

Chess Board State Recognition from Real-World Images

Deep Learning Project Report

Yonatan Arbel, Guy Shick, Tomer Tocker

April 2026

1. Introduction

This project addresses the problem of automatic chess board state recognition from real-world images. Given a single static photograph of a physical chessboard, the system must identify the piece (or absence thereof) on each of the 64 squares and produce a complete board state in Forsyth-Edwards Notation (FEN). The task is challenging due to varying lighting conditions, camera angles, piece styles, and partial occlusions caused by hands or other objects during play.

The system must handle 14 distinct classes: six white piece types (P, N, B, R, Q, K), six black piece types (p, n, b, r, q, k), empty squares, and occluded squares where the content cannot be determined. A key constraint is that classification must operate on single static frames without temporal context from adjacent frames.

2. Data Collection and Preprocessing

The dataset consists of 10 recorded chess games (games 2, 4-12), each containing video frames of a physical chessboard captured from an overhead camera. Each game is accompanied by a CSV file mapping frame numbers to ground-truth FEN strings representing the board state at that moment.

2.1 Offline Data Augmentation

To increase dataset diversity and robustness, each game was augmented offline with four variants: brightness-adjusted, darkened, color-shifted, and noise-injected versions. This expanded the effective dataset from the original game frames to five variants per game (original plus four augmented). The total dataset comprises 2,980 board images across all games and their augmented variants.

2.2 Occlusion Annotation

Occluded squares (caused by hands, heads, or other obstructions) were manually annotated using a custom interactive tagging tool. The tool displays each frame with a grid overlay, allowing annotators to click on occluded squares. Annotations are stored as comma-separated square coordinates (e.g., 'f8,g8,h8') in the CSV. A propagation script ensures occlusion tags are consistently applied across all augmented variants of each frame.

2.3 Board Detection and Square Extraction

Each board image undergoes perspective correction using OpenCV to produce a canonical top-down view. The corrected board is then divided into 64 individual square images, each extracted with 70% overlap padding to provide surrounding context. Squares are resized to 224x224 pixels for input to the classifier.

2.4 Train/Validation Split

The dataset is split by randomly selecting 20% of unique (game_id, frame_number) groups for validation. This frame-group split ensures that augmented variants of the same frame always appear in the same split, preventing data leakage. The resulting split yields 2,385 training boards and 595 validation boards, with validation samples drawn from 9 of the 10 games for representative coverage.

3. Model Architecture

The classifier is built on an EfficientNet-B2 backbone pretrained on ImageNet. The original classification head is replaced with a custom head consisting of Dropout(0.3) followed by a linear layer mapping the

1,408-dimensional feature vector to 14 output classes. The model processes all 64 squares of a board in a single forward pass, producing a (batch, 64, 14) tensor of logits.

3.1 Gradual Unfreezing

Training employs a gradual unfreezing strategy. For the first three epochs, 60% of the backbone parameters (179 of 299 parameter groups) remain frozen, allowing only the classifier head and upper backbone layers to adapt. At epoch 4, all layers are unfrozen and the learning rate is halved from 5e-5 to 2.5e-5. This approach prevents catastrophic forgetting of pretrained features while allowing full network adaptation.

3.2 Loss Function and Class Balancing

The model is trained with cross-entropy loss incorporating label smoothing ($\alpha=0.1$) to prevent overconfident predictions and improve calibration. Class imbalance is addressed through inverse-frequency class weights applied to the loss function. Additionally, a `WeightedRandomSampler` ensures balanced representation during training by computing board-level sampling weights as the mean inverse-frequency of all 64 squares on each board.

4. Training Procedure

Training uses the AdamW optimizer with an initial learning rate of 5e-5 and weight decay of 1e-3. Gradient clipping ($\text{max_norm}=1.0$) is applied for training stability. A `ReduceLROnPlateau` scheduler monitors validation accuracy with patience of 3 epochs and a reduction factor of 0.5. Early stopping with patience of 5 epochs is also employed.

4.1 Online Data Augmentation

During training, online augmentations are applied to individual squares: `ColorJitter` (brightness=0.1, contrast=0.1), random affine transformations (rotation up to 5 degrees, translation up to 3%), `RandomGrayscale` ($p=0.1$), `GaussianBlur` ($p=0.2$), and `RandomErasing` ($p=0.25$). These augmentations are intentionally mild since the offline variants already provide substantial diversity in lighting and noise.

5. Training Evolution

The model went through four major iterations, each building on insights from the previous run.

Run	Key Change	Val Acc	Epochs
1	Full fine-tune, game-level split	72.54%	8 (killed)
2	60% frozen, frame-group split	91.37%	15
3	Gradual unfreezing at epoch 4	94.97%	25
4	Extended training (30 epochs)	95.37%	30

Table 1: Training run evolution and key changes.

Run 1 suffered from severe overfitting (23-point train-val gap) due to full fine-tuning with a high learning rate and an unrepresentative validation set of only 200 samples from 2 games. Run 2 addressed this by freezing 60% of the backbone and introducing the frame-group split, achieving 91.37%. Run 3 introduced gradual unfreezing, bringing accuracy to 94.97% in 25 epochs. Run 4 extended training to 30 epochs, reaching the final 95.37% with no signs of overfitting.

6. Results

6.1 Final Model Performance (Run 4, 30 Epochs)

Metric	Value
Validation Accuracy	95.37%
Training Accuracy	94.74%
Overfit Gap	-0.62% (no overfitting)
Validation Loss	1.8664
Optimal OOD Threshold	0.1
Calibrated Temperature	1.2444

Table 2: Final model performance metrics.

6.2 Per-Class Accuracy

Class	Correct	Total	Accuracy
P (white pawn)	3,090	3,240	95.37%
N (white knight)	621	665	93.38%
B (white bishop)	562	600	93.67%
R (white rook)	814	840	96.90%
Q (white queen)	300	335	89.55%
K (white king)	549	580	94.66%
p (black pawn)	2,873	3,020	95.13%
n (black knight)	465	500	93.00%
b (black bishop)	639	665	96.09%
r (black rook)	820	830	98.80%
q (black queen)	311	335	92.84%
k (black king)	548	580	94.48%
empty	24,193	25,280	95.70%
occluded	531	610	87.05%

Table 3: Per-class accuracy on the validation set.

The model achieves strong performance across all piece types, with black rooks at the highest accuracy (98.80%) and the occluded class at the lowest (87.05%). Queens are the most frequently misclassified pieces (white 89.55%, black 92.84%), likely due to confusion with rooks and bishops. The occluded class is inherently the hardest since it represents visually ambiguous regions.

6.3 Game 2 Evaluation

To compare with related work that uses game-level splits with game 2 as the validation set, we evaluated our trained model on game 2 exclusively. The model achieved 99.35% accuracy on game 2's 24,640 squares (385 boards), with perfect accuracy (100%) on six classes: white bishop, white queen, black knight, black bishop, black rook, and black queen.

7. Out-of-Distribution Detection and Post-Processing

The system employs a hybrid OOD detection strategy combining a supervised occlusion class with confidence thresholding. The model explicitly predicts occluded squares as class 13, while squares with low softmax confidence (below the optimized threshold of 0.1) are additionally flagged as unknown.

Temperature scaling is applied to model logits before computing confidence scores. The temperature parameter ($T=1.24$) is optimized on the validation set using L-BFGS to minimize negative log-likelihood, producing better-calibrated probability estimates.

Chess rule post-processing enforces domain constraints on the predicted board state: no pawns are allowed on ranks 1 and 8, exactly one king per side must be present, and piece counts cannot exceed the starting position limits. These constraints correct a small number of predictions that violate chess rules.

8. Discussion

The training evolution demonstrates the critical importance of proper regularization and data splitting in transfer learning. The jump from Run 1 (72.54%) to Run 2 (91.37%) came primarily from two changes: freezing backbone layers to prevent overfitting, and switching to a frame-group split that provides representative validation data. The further improvement to Run 3 (94.97%) validates gradual unfreezing as an effective strategy for fine-tuning pretrained models on small datasets.

A notable observation across all runs is that validation accuracy consistently exceeds training accuracy. This is expected behavior when dropout is active during training (reducing effective model capacity) while being disabled during validation. Label smoothing further suppresses training accuracy by preventing the model from making perfectly confident predictions on training samples.

The occluded class remains the primary weakness (87.05%), which is expected since occlusions are inherently ambiguous. Queens also show lower accuracy, likely due to visual similarity with rooks (both are tall pieces) and bishops. Future work could explore attention mechanisms to better capture piece shape details, or ensemble methods to reduce confusion between similar piece types.

9. Conclusion

We presented a deep learning system for chess board state recognition that achieves 95.37% validation accuracy using an EfficientNet-B2 backbone with gradual unfreezing. The system handles 14 classes including an explicit occluded class and incorporates confidence calibration, OOD detection, and chess rule post-processing. Evaluation on game 2 yielded 99.35% accuracy, demonstrating strong generalization. The iterative development process, documented across four training runs, highlights the importance of appropriate regularization strategies and representative validation splits when fine-tuning pretrained models on domain-specific tasks with limited data.